

RESEARCH AND IMPLEMENTATION REPORT

Forge: Three Algorithmic Extensions Beyond the Current Design

Inductive subspaces, boundary-aware telescoping,
and two-sided noncommutative certificates

Prepared for Vladimir Reshetnikov
September 15, 2026

The practical recommendation. Keep Forge’s existing orchestration design. Add three narrow workers that discover mathematical structure the current merged proposal does not supply algorithmically: invariant spaces closed under transitions, hypergeometric antidifferences and telescopers, and consequences of noncommutative relations with multiplication on both sides.

Delivered evidence. Executed Python search and independent exact replay for 65 stored certificates; 188 rejected certificate mutations; 1,100 assertions in the generated experiment; 13 named regression tests; reproducible source, problem data, certificates, and logs. The invariant ablation solves 24 of 24 selected cases, versus 3 of 24 for the same-feature conservation-only reference.

Verification boundary. These are research prototypes, not a completed Lean tactic. No Lean compiler was available in this execution environment. Included Lean files are uncompiled replay targets, not kernel-checked results. In particular, polynomial telescoper replay is not silently promoted to a proof of a finite sum with singular endpoints.

Source baseline: Forge commit 674521027d968d59f7b83220ed52304a85cb55e2.

The additions are relative to the inspected design, not claims of new invention of invariant theory, Gosper’s algorithm, or noncommutative ideal theory.

Abstract

The existing Forge repository already contains an unusually broad architecture: obligation-directed search, induction planning, exact arithmetic certificates, witness synthesis, selective equality exploration, and carefully delimited external search. Repeating that architecture would not make the tactic more capable. This report develops three additional, executable mathematical workers. First, a descending sequence of rational linear subspaces discovers families of polynomial invariants satisfying $p(T(x)) = Hp(x)$, allowing scaling and mutual dependence rather than requiring each polynomial to be conserved. It terminates by dimension descent and is complete for bounded-degree polynomial equalities of the specified affine, nondeterministically branching fragment. Second, coefficient-linear searches discover hypergeometric antidifferences and an order-one creative telescoper. A worked squared-binomial example makes the singular-boundary problem explicit and derives a nonsingular flux, so that algebraic certificate checking and finite-sum semantics remain distinct. Third, bounded two-sided relation search in a free associative algebra emits identities $f = \sum curv$, enabling matrix and operator reasoning without assuming commutativity or finding a terminating rewrite orientation. Each worker comes with a concrete search algorithm, a separate exact checker, mathematical soundness arguments, sharply bounded completeness claims, and a Lean integration route. The accompanying experiments include positive and negative controls, adversarial mutations, an invariant ablation, and standard-library-only replay in a fresh Python process. The report also explains how these workers can strengthen universal behavioral checking of exact Leant/Djex candidates without redesigning their existing typed streams.

Contents

1	Recommendation and scope	2
2	Inductive subspaces instead of conserved scalars	3
3	Hypergeometric antidifference certificates	8
4	Creative telescoping with a checked boundary plan	10
5	Two-sided certificates for noncommutative algebra	13
6	A stronger universal-assertion backend for Leant and Djex	17
7	Lean integration: concrete modules and library choices	20
8	Executed experiments and what they establish	22
9	Implementation sequence and acceptance criteria	24
10	Conclusion	25
A	Certificate formats and reproducibility	26
B	A compact set of regression targets for a Lean port	27

1 Recommendation and scope

1.1 Add mathematical discovery, not another controller

Forge’s next useful increment should be a small number of workers with a clear answer to four questions: what do they discover, what finite object do they return, what does its checker establish, and how does the resulting theorem help a blocked goal? The three proposed workers answer those questions without replacing the existing obligation graph or inventing a second policy for proof admission.

The invariant worker discovers *relations preserved collectively*. The summation worker discovers *an antidifference or a recurrence with a local flux*. The noncommutative worker discovers *which left and right contexts make the given relations add up to the target*. These are different ways of finding an intermediate proof object, not merely running a familiar tactic with more fuel.

The intended deployment is initially conservative. A worker receives a supported problem extracted from the actual Lean goal. Search proposes a certificate. Ordinary Lean theorem applications and existing normalizers reconstruct its small algebraic identities. The new local theorem is then given to Forge’s existing machinery. A reflected checker can replace large proof scripts later; there is no need to wait for a new kernel-facing plugin architecture before measuring the first useful cases.

1.2 What was actually inspected

The review is pinned to Forge commit 674521027d968d59f7b83220ed52304a85cb55e2, Leant commit 6bf05ad78c467989e68290f2d08bbcd40802d485, and Djex commit e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5. The Forge README, repository tree, merged structural, nonlinear, Boolean, integration, and provenance sections, and its Leant/Djex review supplied the baseline. The neighboring projects’ behavioral-synthesis and streaming reports were also read at the listed revisions [1, 2, 3, 4, 5, 6, 7, 8].

A complete local clone was not available. Targeted repository searches for summation-related and noncommutative terms returned no matches, but search absence is not a proof that no historical file contains an adjacent idea. Consequently the nonduplication claim is deliberately scoped: the algorithms below extend the *inspected merged design and its provenance ledger*. This is not a line-by-line review of every original proposal or of all Haskell implementation files.

At the reviewed revision, Forge describes a design and a collection of prototypes, not an implemented all-purpose Lean tactic. The repository’s own Lean experiments are distinct from the experiments delivered here. Its baseline identifies Lean v4.34.0 and Mathlib commit 1cf325a0cf67aca2b04d76b5380ff6a9e410aefa; this report reuses those as *integration targets*, not as locally tested dependencies [1, 5].

1.3 A nonduplication ledger

The controller, Horn slicing, induction-variable selection, accumulator generalization, anti-unification, SOS/Bernstein/Sturm machinery, Farkas and lattice witnesses, CDCL(T), basic LJT proposals, and generic security boundaries are not reintroduced as contributions. They are infrastructure or neighboring workers that these additions can use [2, 3, 4, 5].

Area	Already in the inspected design	Increment developed here
Inductive equations	Polynomial finite-difference CEGIS; conserved state quantities; a warning that jointly unknown invariant ideals lead to bilinear search	A dimension-descending subspace algorithm, transition matrices, finite-fragment completeness, and a same-feature ablation
Finite sums	Additive polynomial recurrences and exact coefficient checking	Rational hypergeometric multipliers; an explicitly searched bivariate telescope; nonsingular flux and boundary obligations
Algebra	Commutative polynomial cones, equality ideals, and selective rewriting/e-graphs	Two-sided consequences in a free associative algebra, word order preserved, with coefficient-linear search and relation witnesses
Leant/Djex transfer	Typed streams, exact-candidate verification, evidence ownership, fairness, and negative-evidence restrictions	A specific universal behavioral-proof backend and product-state extraction contract for eligible candidates

1.4 Four evidence levels used throughout

A *mathematical theorem* below has an ordinary written proof. An *executed algorithm* has a runnable implementation and a recorded result. An *exact replay* has been checked by the supplied Python checker against the separately supplied source problem. A *Lean proof* would additionally require successful elaboration and kernel checking at the actual pin. Only the first three levels were attained in this run. Source code ending with `by ring` is not reported as a fourth-level result merely because it looks plausible.

This distinction is especially important for telescoping: the current checker accepts an identity in a rational polynomial ring. The theorem connecting that identity to a source sequence, its index domain, and its boundary conditions must still be proved. The experiment does not disguise this gap with a misleading universal PROVED status.

2 Inductive subspaces instead of conserved scalars

Evidence status. General finite-feature search and exact closure replay are implemented. Twenty-four positive cases, three expected-unknown cases, a same-feature conservation ablation, and corruption tests were executed. Lean extraction and a generic reflected checker are not implemented.

2.1 The capability gap

A conserved quantity satisfies $p(T(x)) = p(x)$. An inductive relation needs much less: whenever $p(x) = 0$, its updated value must also be zero. For example,

$$T(x, y, z) = (2x, 3y, 6z), \quad p(x, y, z) = z - xy$$

satisfies $p(T(x, y, z)) = 6p(x, y, z)$. Starting at (u, v, uv) , the relation $z = xy$ remains true after every number of steps. A search restricted to $p \circ T = p$ throws this relation away for a reason unrelated to its validity.

There is a second gap: a useful family can be preserved while none of its chosen basis elements is preserved in isolation. Under $T(x, y) = (y, 2x)$, the family (y, x) transforms by

$$\begin{pmatrix} y \circ T \\ x \circ T \end{pmatrix} = \begin{pmatrix} 0 & 2 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} y \\ x \end{pmatrix}.$$

Both vanish forever from $(0, 0)$, although a scalar conservation test finds no nonzero linear quantity of that kind.

Forge’s structural section already recognizes the general invariant problem and warns that simultaneously solving for p_i and arbitrary polynomial multipliers in $p_i \circ T = \sum_j h_{ij}(x)p_j$ is bilinear. That warning is correct. The algorithm here does not rename that bilinear problem “linear.” It chooses a different, useful fragment and computes its greatest fixed subspace by successive linear problems [2].

2.2 Problem model

Let K be an infinite field of characteristic zero; the prototype calculates coefficients in $\mathbb{Q}$. State variables are $x = (x_1, \dots, x_n)$. Initial states are specified by a polynomial map

$$I : K^p \longrightarrow K^n,$$

including $p = 0$ for a single fixed state. A finite nonempty collection of polynomial maps $T_b : K^n \rightarrow K^n$ gives the permitted updates. Every branch is initially treated as nondeterministic: it may be taken at any reachable state.

Reachable states are obtained from some $I(u)$ by a finite composition of the T_b . A guard can be ignored to obtain a sound overapproximation; it cannot be silently used as a proof assumption. A guard-sensitive refinement is discussed later. There is no assertion here that the transition system terminates.

Choose a finite-dimensional rational space

$$E = \text{span}_{\mathbb{Q}}\{m_1, \dots, m_N\} \subseteq \mathbb{Q}[x].$$

The implementation uses all monomials of total degree at most D , so $N = \binom{n+D}{D}$. The mathematical construction also works for a selected feature list after removing linear dependencies. The pullback $T_b^*p = p \circ T_b$ is linear in p , even when T_b is nonlinear. This observation supplies the useful linear structure.

2.3 Descending closure

Define

$$V_0 = \{p \in E : p \circ I = 0 \text{ as a polynomial}\}, \tag{1}$$

$$V_{j+1} = V_j \cap \bigcap_b (T_b^*)^{-1}(V_j). \tag{2}$$

The inverse-image condition means that $p \circ T_b$ belongs to the current subspace as a polynomial, not merely after discarding high-degree monomials. At each iteration the current basis is fixed; only the coefficients of a new candidate and its coordinates in that fixed basis are unknown. These are linear constraints.

Theorem 2.1 (Termination and maximality). *The sequence (2) stabilizes after at most N strict dimension drops. Its final space V_* is the largest subspace of E that vanishes under initialization and is stable under every pullback T_b^* . Every polynomial in V_* vanishes at every reachable state.*

Proof. Each V_j is a linear subspace, and $V_{j+1} \subseteq V_j$. A strict inclusion between finite-dimensional spaces strictly decreases dimension, so there can be at most N such inclusions. Equal dimensions at a step imply equality of spaces and therefore a fixed point.

Suppose $W \subseteq E$ vanishes under initialization and $T_b^*W \subseteq W$ for every branch. Then $W \subseteq V_0$. If $W \subseteq V_j$, every $p \in W$ also has $T_b^*p \in W \subseteq V_j$, whence $p \in V_{j+1}$. Induction gives $W \subseteq V_*$, proving maximality.

At an initial state every $p \in V_*$ vanishes. At a transition from a state where all members of V_* vanish, each $p \circ T_b$ is a linear combination of members of V_* . It therefore vanishes too. Induction on the finite execution length proves the last assertion. $\square$

The conclusion does not require the returned basis to be linearly independent for *soundness*. Independence is a search and size property; a checker can accept a redundant family if all displayed identities are valid.

2.4 Exact matrix construction

The coefficient representation must include both the original features and all their pullbacks. Let $\mathcal{M}$ be the union of monomial supports of E and T_b^*E for every branch, and let $M = |\mathcal{M}|$. In this common ambient coordinate space, write

$$A_E \in \mathbb{Q}^{M \times N}, \quad A_b \in \mathbb{Q}^{M \times N}$$

for the columns representing m_i and $m_i \circ T_b$ respectively. Write C_I for the coefficient matrix of the $m_i \circ I$. Then a basis matrix for V_0 is a nullspace basis of C_I .

Suppose $C_j \in \mathbb{Q}^{N \times d_j}$ has columns spanning V_j . Compute a matrix L_j whose rows span the left nullspace of $A_E C_j$. Membership in that column space is equivalent to annihilation by L_j . Therefore

$$C_{j+1} = C_j \text{ basisKer} \begin{pmatrix} L_j A_I C_j \\ \vdots \\ L_j A_B C_j \end{pmatrix}. \quad (3)$$

All matrices in a single call to `basisKer` are already known. There are no products of two independently unknown coefficient families.

```

features := monomials(state_variables, degree_cap)
C := kernel(coefficients(features after initialization))
ambient := supports(features and every branch pullback)
AE, Ab := coefficient_matrices_in_that_same_ambient
repeat:
  L := transpose(kernel(transpose(AE * C)))
  K := kernel(stack_b(L * Ab * C))
  Cnext := independent_columns(C * K)
  if columns(Cnext) == columns(C): return Cnext
  C := Cnext

```

The ambient support is *not* recomputed by truncating pullbacks to degree D . Such truncation would make $x \mapsto x^2$ look like the zero map on the space spanned by x and would destroy the meaning of the test. Simultaneous substitution is equally important: replacing (x, y) by (y, x) must not become two successive substitutions that accidentally identify both variables. The implementation and regression suite test this explicitly.

2.5 Certificate and independent checker

If the final basis is $p = (p_1, \dots, p_r)^\top$, search returns a matrix $H_b \in \mathbb{Q}^{r \times r}$ per branch and a vector $a \in \mathbb{Q}^r$ for a requested target g , with

$$p_i(I(u)) = 0, \tag{4}$$

$$p(T_b(x)) = H_b p(x), \tag{5}$$

$$g(x) = a^\top p(x). \tag{6}$$

The checker obtains I, T_b, g from the separately provided problem. It expands (4)–(6) exactly. It neither reruns the fixed-point algorithm nor accepts a target polynomial embedded by the producer as a replacement for the source target.

Corollary 2.2 (Certificate soundness). *An accepted certificate proves $g(s) = 0$ at every reachable state of the supplied transition system.*

Proof. The initialization and transition identities prove that all components of p vanish by induction. Equation (6) gives the target. $\square$

The prototype producer uses SymPy expressions, polynomial coefficient matrices, and exact nullspaces. The checker instead uses sparse exponent tuples and Python’s standard-library `Fraction`. This separates matrix construction and solving from checking the result. Both programs still depend on the Python runtime and rational arithmetic; algorithmic independence is not a formal verification theorem.

2.6 A useful completeness theorem, and its exact boundary

Theorem 2.3 (Bounded-degree affine completeness). *Suppose every T_b is affine, E contains all monomials of total degree at most D , and initial states are exactly the image of the supplied polynomial map $I : K^p \rightarrow K^n$. Ignoring resource limits, V_* consists of precisely the rational polynomials of degree at most D that vanish at every reachable state.*

Proof. An affine substitution maps a polynomial of degree at most D to one of degree at most D , so every pullback preserves E . Let W be the space of members of E vanishing on the reachable set. It vanishes on every initial state. Because K is infinite, a polynomial in the initial parameters vanishing at all parameter values is the zero polynomial; hence $W \subseteq V_0$. If $p \in W$, then $p(T_b(s)) = 0$ for every reachable s , since its successor is reachable. Thus $T_b^* p \in W$. Maximality gives $W \subseteq V_*$. Soundness gives the reverse inclusion. $\square$

This is not the strongest known result about affine programs. Algorithms for strongest algebraic invariants, without this fixed degree-space restriction, are established prior work [9]. The merit

here is the small, executable fragment and its cheap identity certificate, not a claim to have solved invariant generation for the first time.

For a selected feature set not closed under pullback, the maximality theorem still holds but affine completeness may not. For nonlinear transitions the failure can be unavoidable at every finite degree.

Example 2.4 (A nonlinear invariant that no finite closed subspace sees). Let $T(x) = x^2$, initialize $x = 0$, and take $E = \mathbb{Q}[x]_{\leq D}$ for any finite positive D . The reachable state is always zero, so $x = 0$ is an invariant. Nevertheless, any nonzero pullback-stable subspace vanishing at zero would contain a polynomial of maximal positive degree d . Its pullback has degree $2d > d$, a contradiction. Hence $V_* = 0$ for every finite D . The ordinary ideal certificate $x \circ T = x \cdot x$ succeeds, but its multiplier is polynomial rather than constant.

The right response is to route this case to a candidate-fixed multiplier search or a different induction argument, not to advertise arbitrary invariant completeness. If a candidate polynomial has already been selected, searching for polynomial multipliers is again linear in those multipliers. Discovering both sides at once remains the harder problem already identified by Forge.

2.7 Worked discoveries and proof economy

For the scaling example the run returned basis $(xy - z)$, transition matrix (6), and target coefficient -1 . That complete certificate occupies 121 bytes under the experiment’s compact JSON convention. The different sign from the motivating polynomial is immaterial: the checker verifies the actual basis and target combination rather than relying on an expected spelling.

For $T(x, y) = (y, 2x)$, the basis was (y, x) and the matrix was $\begin{pmatrix} 0 & 2 \\ 1 & 0 \end{pmatrix}$. The selected target x uses the coordinate vector $(0, 1)$. This demonstrates why a family-valued certificate is more natural than independent scalar certificates.

For

$$(x, y)_0 = (0, 0), \quad T(x, y) = (x + 1, y + 2x + 1),$$

the six-dimensional quadratic feature space yields dimensions

$$5 \longrightarrow 4 \longrightarrow 3 \longrightarrow 2 \longrightarrow 1 \longrightarrow 1.$$

The surviving basis is $(x^2 - y)$. This example is intentionally easy to prove once the relation is known; it tests the discovery process and the stabilization criterion. Unlike fitting a polynomial through a few observed states, the final step checks closure on every state symbolically.

2.8 Guards, multiple locations, and implementation limits

Ignoring guards preserves soundness because it adds behaviors. It may lose nearly every interesting invariant. The experiment includes an initially zero counter with transition $x \mapsto x + 1$ and an intended guard $x < 0$. Forgetting the guard makes $x = 0$ fail, although the guarded system cannot move. The worker returns UNKNOWN; it does not refute the guarded source theorem.

A multi-location control-flow graph admits a direct extension: keep a space V_ℓ at each location, intersect an edge source space with the inverse image of its destination space, and impose initialization only at entry locations. The sum of the location-space dimensions decreases at every strict update,

giving termination. A certificate has a basis per location and a closure matrix per edge. This extension is specified here but not implemented in the delivered single-location prototype.

The implementation caps the monomial feature count at 256. There are at most N strict descent steps, each involving exact elimination on matrices built from N features and M ambient monomials. Polynomial-time rational-operation counts do not imply small bit complexity: coefficients can grow sharply. For production, exploit sparse supports, cache branch pullbacks, and substitute an exact matrix backend such as SymPy’s `DomainMatrix` with rational or fraction-free row reduction [10]. A target-guided feature set can be cheaper, but its weaker completeness claim must travel with it.

3 Hypergeometric antidifference certificates

Evidence status. A denominator-enumerating, coefficient-linear search and an independent rational-identity checker are implemented. Fifteen positive cases and two expected-unknown cases were run. A separate SymPy Gosper adapter passed four regression inputs. A generic Lean sequence-to-ratio bridge is not implemented.

3.1 From polynomial increments to multiplicative structure

The polynomial finite-difference worker already in Forge is appropriate when $F(n+1) - F(n)$ is polynomial. It does not, by itself, discover the useful structure in factorials, geometric terms, or rational products. For example,

$$\sum_{k=a}^{b-1} k k! = b! - a! \quad (1 \leq a \leq b)$$

is a one-line proof after recognizing an antidifference, but its summand is not a polynomial in k . The new worker searches for an antidifference expressed as a rational multiple of the summand.

A hypergeometric term has a rational consecutive ratio. For proof purposes it is better to encode that ratio without dividing by the term itself:

$$B(k)t(k+1) = A(k)t(k), \quad A, B \in \mathbb{Q}[k]. \quad (7)$$

The source layer must prove this recurrence on the indices where it will be used. A zero value of $t(k)$ is not automatically a problem; an unjustified cancellation of $t(k)$ would be.

Seek $G(k) = R(k)t(k)$ with $G(k+1) - G(k) = t(k)$, writing $R = U/V$. Clearing denominators gives the exact polynomial identity

$$A(k)U(k+1)V(k) - B(k)U(k)V(k+1) = B(k)V(k)V(k+1). \quad (8)$$

For a fixed candidate denominator V and a degree bound on U , the coefficients of U enter linearly. This is the first useful implementation: exact algebra, a small certificate, and no floating-point recovery.

3.2 Search and fixed-ansatz completeness

For every admitted nonzero V , enumerate numerator degrees $0, \dots, d$. Substitute $U(k) = \sum_{i=0}^d u_i k^i$ into (8), equate coefficients, and solve the rational linear system. A solution yields a proposed pair (U, V) .

```

for V in nonzero_denominator_candidates:
    for numerator_degree in 0 .. degree_cap:
        U := polynomial_with_unknown_coefficients(numerator_degree)
        residual := A * shift(U) * V - B * U * shift(V)
                - B * V * shift(V)
        coefficients := exact_linear_solve(coefficients(residual) = 0)
        if a solution exists:
            return U[coefficients], V
return UNKNOWN

```

Exact elimination is complete for each fixed finite numerator space and fixed V . Enumerating a handful of denominators is not a complete implementation of Gosper's algorithm. In the delivered bounded search, unsuccessful enumeration returns UNKNOWN even when the mathematical nonexistence of an antidifference may be known by some stronger method.

Production should use the established algorithm rather than turn the hand-selected denominator list into a permanent limitation. SymPy exposes `gosper_normal`, `gosper_term`, and `gosper_sum`; these are concrete, available search interfaces [11]. The delivered `sympy_gosper_oracle` obtains the consecutive ratio with `hypersimp`, obtains the proposed multiplier with `gosper_term`, and converts the numerator and denominator to the same certificate format. It refuses non-polynomial residual components. The independent identity checker, not the API's naming convention, decides whether the returned object has the required multiplier meaning.

On the locally tested SymPy 1.14.0, the outputs for $kk!$, $1/(k(k+1))$, 2^k , and $k2^k$ were respectively $1/k$, $-k-1$, 1 , and $(k-2)/k$. Each passed (8). These four adapter checks are reported separately from the 65 stored corpus certificates; they are not four additional stored results.

3.3 The theorem that connects the identity to a sum

Theorem 3.1 (Antidifference lifting). *Let t take values in a characteristic-zero field K . At a particular index k , suppose (7) holds and $B(k)$, $V(k)$, and $V(k+1)$ are nonzero. If (8) is a polynomial identity, then*

$$\frac{U(k+1)}{V(k+1)}t(k+1) - \frac{U(k)}{V(k)}t(k) = t(k).$$

Proof. Multiply the proposed equality by the nonzero product $B(k)V(k)V(k+1)$. The left side becomes

$$B(k)U(k+1)V(k)t(k+1) - B(k)U(k)V(k+1)t(k).$$

Use (7) in the first term, factor out $t(k)$, and apply (8). The result is the right side multiplied by the same nonzero product. Cancel that product. No cancellation of $t(k)$ was used. $\square$

Corollary 3.2 (Finite telescoping). *If the hypotheses of Theorem 3.1 hold for every integer k with $a \leq k < b$, then*

$$\sum_{k=a}^{b-1} t(k) = G(b) - G(a), \quad G(k) = \frac{U(k)}{V(k)}t(k).$$

The empty range $a = b$ has sum zero.

Proof. Replace each summand by $G(k+1) - G(k)$ and cancel adjacent terms, or induct on the number of summands. The endpoint values are defined by the same justified formula or separately proved extensions. $\square$

The Python checker validates only (8) and its syntactic well-formedness. It does not establish the source recurrence, the quantified nonvanishing conditions, or the source indexing convention. Those are inputs to the lifting theorem, not consequences of a successful algebraic calculation.

In Lean, natural-number subtraction is truncated and field division is total. Thus translating a denominator or a shifted index by surface syntax alone is unsafe. Prove the integer or rational cast identities, obtain sign or nonvanishing facts using the source bounds, and only then invoke the antidifference theorem. The finite range must also be explicit: Mathlib’s `Finset.range N` excludes N , whereas SymPy’s displayed finite-sum bounds are inclusive and its generalized reversed-bound conventions are not Lean finite-set semantics [12, 11].

3.4 Three representative discoveries

For $t(k) = kk!$, the recurrence has $A = (k+1)^2$ and $B = k$. The worker tries constant-denominator numerators through its bound, then finds

$$U = 1, \quad V = k, \quad R = 1/k, \quad G(k) = k! \quad (k \geq 1).$$

The identity check succeeds with a 65-byte compact certificate. Extending this formula blindly to $k = 0$ would replace an intended factorial antidifference by an undefined rational expression before Lean totalizes it. Either work on $k \geq 1$ or first prove the cancellation and use the nonsingular expression $G(k) = k!$ on the larger domain.

For $t(k) = 1/(k(k+1))$, use $A = k$, $B = k+2$. The worker finds $R = -(k+1)$, so $G(k) = -1/k$ on $k \geq 1$. The familiar reciprocal telescoping formula is therefore recovered through the same certificate language as the factorial example, not by a special-purpose syntactic rewrite.

For $t(k) = r^k$ with fixed nonzero rational $r \neq 1$, the result is $R = 1/(r-1)$. For $t(k) = kr^k$, a valid multiplier is

$$R(k) = \frac{1}{r-1} - \frac{r}{k(r-1)^2}.$$

At $k = 0$ this multiplier is singular, although the product can often be rewritten to a nonsingular G . Treat that cancellation as a separate algebraic lemma rather than declaring the guard unnecessary because a numerical test happened to pass.

The corpus includes six rational choices of r for each family, including a negative base and $r = 1/2$. It also keeps bounded-search failures for the factorial and harmonic summands. These are expected UNKNOWN outcomes, not theorem refutations.

4 Creative telescoping with a checked boundary plan

Evidence status. An order-one, six-unknown coefficient search is implemented and returns an exact polynomial certificate. The nonsingular boundary argument is derived mathematically below and checked on 527 concrete index pairs. No general Lean creative-telescoping adapter or universal binomial-sum proof was run.

4.1 Search for a recurrence, not a closed form directly

A sum may have no convenient hypergeometric antidifference in its summation variable, while its dependence on a parameter satisfies a short recurrence. For a bivariate term $t(n, k)$, seek

$$c_0(n)t(n, k) + c_1(n)t(n+1, k) = G(n, k+1) - G(n, k), \quad G(n, k) = R(n, k)t(n, k). \quad (9)$$

Summing can then remove the right side. More generally one can admit several shifts in n ; the delivered prototype uses order one only.

Suppose the two rational ratios are A_n/B_n and A_k/B_k , and write $R = U/V$. The independently checked polynomial identity is

$$(c_0 B_n + c_1 A_n) B_k V V^{+k} = B_n (A_k U^{+k} V - B_k U V^{+k}), \quad (10)$$

where the superscript $+k$ shifts only k . The denominator template V is fixed during search, and unknown coefficients in c_0, c_1, U enter linearly. The checker explicitly rejects recurrence coefficients containing k ; such coefficients could not simply be taken outside the sum.

This is a concrete entry point for a later Ore-algebra worker. Sage's optional `ore_algebra` package documents creative telescoping, rational-solution methods, and desingularization [13]. It is a search dependency, not a Lean theorem library. A returned recurrence alone is insufficient: an adapter must obtain a telescoping certificate or another independently replayable derivation, together with a domain and boundary argument.

4.2 Discovering the squared-binomial recurrence

Take

$$t(n, k) = \binom{n}{k}^2, \quad S(n) = \sum_{k=0}^n \binom{n}{k}^2.$$

On the nonsingular interior the ratios are

$$\frac{t(n+1, k)}{t(n, k)} = \frac{(n+1)^2}{(n-k+1)^2}, \quad \frac{t(n, k+1)}{t(n, k)} = \frac{(n-k)^2}{(k+1)^2}.$$

Use the ansatz

$$c_0 = an + b, \quad c_1 = n + c, \quad U = k^2(dn + ek + f), \quad V = (n - k + 1)^2.$$

Fixing the coefficient of n in c_1 to one removes the irrelevant overall scaling for this selected template. It does not make the search complete over all telescopers.

Equating coefficients in (10) produced 48 equations in six unknowns, of rank six. The discovered solution was

$$c_0 = -4n - 2, \quad c_1 = n + 1, \quad R(n, k) = \frac{k^2(2k - 3n - 3)}{(n - k + 1)^2}. \quad (11)$$

The compact certificate is 313 bytes. The checker reproduces the complete bivariate polynomial identity by its own sparse arithmetic.

This is an executed search: the coefficients in (11) were not inserted as a ready-made certificate. The denominator and low-degree template were selected in advance. That template selection is an honest form of domain knowledge, and its cost and limitations should not disappear in a future benchmark.

4.3 Why algebraic replay is not yet a sum proof

The displayed R has a pole at $k = n + 1$. In addition, the shifted denominator $V(n, k + 1)$ vanishes at $k = n$. A rational identity derived where denominators are nonzero cannot be evaluated across these points merely by clearing denominators and then forgetting the guards.

The summand is zero outside its combinatorial support, but “zero times a singular rational expression” is not a justified boundary value. In Lean the division expression has a totalized value; that value need not be the extension required by the telescoping proof. The regression suite includes a test where the algebraic certificate is accepted while its denominator evaluates exactly to zero at the boundary. This is intentional: the algebraic checker is doing its narrower job, not lying about a boundary theorem.

4.4 A nonsingular flux

The ratio between consecutive rows of Pascal’s triangle suggests rewriting Rt before evaluating it at the singular points. Define, for natural n, k ,

$$\tilde{G}(n, k) = \frac{k^2(2k - 3n - 3)}{(n + 1)^2} \binom{n + 1}{k}^2, \quad (12)$$

interpreting the polynomial arithmetic in $\mathbb{Q}$ or $\mathbb{R}$, and setting the binomial coefficient to zero outside its natural support. The only denominator is $(n + 1)^2$, which is nonzero for natural n .

On $0 \leq k < n$, both adjacent flux values agree with the rational certificate where the relevant denominators are nonzero, so

$$(n + 1)t(n + 1, k) - 2(2n + 1)t(n, k) = \tilde{G}(n, k + 1) - \tilde{G}(n, k). \quad (13)$$

The remaining cases $k = n$ and $k = n + 1$ must be proved directly.

At $k = n$,

$$\tilde{G}(n, n) = -n^2(n + 3), \quad \tilde{G}(n, n + 1) = -(n + 1).$$

The left side of (13) is $(n + 1)^3 - 2(2n + 1)$; its right side is $n^2(n + 3) - (n + 1)$. They agree by polynomial expansion, including $n = 0$. At $k = n + 1$, the left side is $n + 1$ because $t(n, n + 1) = 0$ and $t(n + 1, n + 1) = 1$. The right side is also $n + 1$, since $\tilde{G}(n, n + 2) = 0$ and $\tilde{G}(n, n + 1) = -(n + 1)$. Finally $\tilde{G}(n, 0) = 0$.

Theorem 4.1 (Squared-binomial recurrence). *For every natural n ,*

$$(n + 1)S(n + 1) = 2(2n + 1)S(n), \quad S(0) = 1. \quad (14)$$

Consequently $S(n) = \binom{2n}{n}$.

Proof. Equation (13) holds for $k = 0, \dots, n + 1$ by the interior identity and the two direct boundary checks. Sum over precisely that range. The flux telescopes to $\tilde{G}(n, n + 2) - \tilde{G}(n, 0) = 0$. The term $t(n, n + 1)$ vanishes, so the two sums on the left are $(n + 1)S(n + 1)$ and $2(2n + 1)S(n)$. The initial value is immediate.

The central binomial coefficient $C(n) = \binom{2n}{n}$ has $C(0) = 1$ and, from its factorial expression,

$$\frac{C(n + 1)}{C(n)} = \frac{(2n + 2)(2n + 1)}{(n + 1)^2} = \frac{2(2n + 1)}{n + 1}.$$

Induction using the nonzero coefficient $n + 1$ gives $S(n) = C(n)$. □

The theorem is an ordinary mathematical derivation, not a claim that the Python boundary loop proved it. The 527 pointwise checks for $0 \leq n \leq 30$ and $0 \leq k \leq n + 1$, together with 31 sum and 31 flux checks, are independent regression evidence for the formulas and the endpoint indexing.

4.5 A production boundary planner

A usable summation worker should return a plan with four parts: the algebraic identity; a region where the source ratio equations and all denominator guards are proved; a finite list of excluded boundary strata with direct proofs; and a final range-combination proof. The controller may schedule these as separate obligations, but none may be omitted merely because the main identity passed.

The first practical domain is rational and factorial/binomial terms on a single finite natural interval. Factor denominators, solve their integer zero conditions where possible, and split off the finitely many exceptional indices. Then attempt cancellation, a neighboring hypergeometric representation, or a direct recurrence proof for those indices. The squared-binomial example uses the neighboring-row representation (12).

An apparent singularity depending on an unconstrained parameter is not always a finite boundary issue; it may require parameter case splits. A leading recurrence coefficient can also vanish and break uniqueness. The worker must return those cases to the proof planner rather than divide through them. Infinite sums require convergence and limit arguments and are outside this proposal's first implementation.

5 Two-sided certificates for noncommutative algebra

Evidence status. Bounded exact search and an independent word-polynomial checker are implemented. Twenty-five positive instances and two expected-unknown cases were executed. Two integral certificates were emitted as ordinary Lean replay scripts; they were not compiled.

5.1 Relations are more than normal forms

Commutative polynomial reasoning identifies xy and yx at the representation level. That is invalid for matrices, linear operators, and a general associative algebra. An oriented rewrite system avoids that error, but it introduces a different burden: useful relations may not have a convenient terminating orientation, and the required consequence may use a relation in several left and right contexts.

Mathlib already has `noncomm_ring`. At Forge's exact Mathlib pin, its source expands and reassociates expressions using simplification and additive normalization; it supports supplied rewrite lemmas and warns that numeral powers are fully unfolded. It is an appropriate replay tool for small identities, not a reason to claim that noncommutative normalization is missing from Lean [14]. The increment here is systematic *relation consequence discovery* with explicit two-sided witnesses.

5.2 Free associative representation

Let

$$F = \mathbb{Q}\langle X_1, \dots, X_s \rangle$$

be the free associative unital algebra. A monomial is a finite word, including the empty word ε for 1. Multiplication concatenates words; it never sorts letters. A polynomial is a finite rational linear combination of words.

The source problem consists of relation polynomials $r_1, \dots, r_m$ asserted to be zero and a target polynomial f to prove zero. A certificate is a finite list of terms

$$f = \sum_{\ell=1}^q c_\ell u_\ell r_{j_\ell} v_\ell, \quad c_\ell \in \mathbb{Q}, \quad (15)$$

where u_ℓ, v_ℓ are words. The checker expands the right side in the free algebra and compares it with the independently supplied target.

Theorem 5.1 (Two-sided certificate soundness). *Let A be an associative unital $\mathbb{Q}$ -algebra, and let an assignment of the letters to A make every r_j vanish. If (15) holds in the free algebra, the assignment also makes f vanish.*

Proof. Evaluate words by ordered multiplication and extend linearly using the central rational scalars. This is the canonical algebra homomorphism from the free algebra. Each evaluated summand on the right is a scalar multiple of $u_\ell 0 v_\ell$, hence zero. Applying the homomorphism to the identity proves the result. $\square$

Mathlib's `FreeAlgebra` supplies a natural semantic setting for a later reflected checker [16]. For a small first implementation, the same theorem can be instantiated without constructing a generic quotient algebra: prove the displayed identity with `noncomm_ring`, then rewrite each relation occurrence to zero.

5.3 Exact bounded search

Choose a maximum context degree D and enumerate

$$\mathcal{C}_D = \{ur_jv : |u| + \deg(r_j) + |v| \leq D\}. \quad (16)$$

Each member is a column in the rational coefficient space of words of length at most D . Solve the linear membership problem

$$A_D c = f.$$

The implementation inserts columns incrementally into a sparse echelon basis and records every reduced column's linear combination of original dictionary columns. After an independent column is inserted, it tries to reduce the target. When the residual becomes zero, the recorded combination is already the certificate (15).

```

pivots := empty sparse echelon basis
for degree in 0 .. D:
  for relation r and left/right words of the required lengths:
    column := left * r * right
    representation := the corresponding dictionary basis vector
    reduce column using prior pivots, updating representation
    if nonzero:
      normalize its pivot and store both objects
      reduce target through the current pivots
      if target residual is zero:
        return its combination of original columns
return UNKNOWN

```

The zero target is handled separately with an empty certificate, including when there are no relations. This small case matters: a membership algorithm that forgets it cannot claim completeness even for its own finite span.

Proposition 5.2 (Exact finite-span completeness). *Ignoring column and arithmetic resource caps, the bounded worker finds a certificate exactly when $f \in \text{span}_{\mathbb{Q}} \mathcal{C}_D$.*

Proof. Sparse echelon insertion preserves the span of every prefix of input columns. After all columns are processed, the stored pivot columns span the whole dictionary. Exact reduction has zero residual exactly for a vector in that span. The tracked representations express every pivot, and therefore the accepted target, in the original columns. $\square$

This statement is deliberately weaker than “complete for ideal membership of targets of degree at most D .” A low-degree target can require cancellations between higher-degree ideal multiples. Membership in a bounded dictionary does not exhaust such possibilities. If all finite degrees are eventually explored, a particular finite representation will eventually be included, but the prototype has explicit caps and no unbounded termination claim.

5.4 An explicit bound on dictionary growth

For a relation of degree d_j , and an alphabet of size s , the number of word-context pairs with total length e is $(e+1)s^e$: choose the left length, then choose the e letters. Therefore the unreduced dictionary has at most

$$C_D = \sum_{j=1}^m \sum_{e=0}^{D-d_j} (e+1)s^e \quad (17)$$

columns, with a negative upper limit interpreted as an empty sum. The ambient word space has dimension at most $\sum_{e=0}^D s^e$. Thus exact linearity is not a polynomial-in- D scalability claim. Sparse support and early target membership can help enormously, but an unrestricted letter alphabet or degree increase is expensive.

The prototype caps columns at 15,000 and uses degrees up to nine in the main corpus. Target word supports can guide a later enumeration, but any pruning that removes contexts loses the displayed finite-span completeness guarantee unless its admissibility is proved. A practical controller should request a small D , examine the residual and support growth, and escalate only when that information justifies the cost.

5.5 Worked certificate: commuting idempotents

Let $a^2 = a$, $b^2 = b$, and $ab = ba$. Define $e = a + b - ab$. We want $e^2 = e$. Put

$$r_0 = a^2 - a, \quad r_1 = b^2 - b, \quad r_2 = ab - ba.$$

The search found the following eight-term certificate:

$$e^2 - e = r_0 + r_1 - r_0b - br_0 - ar_1 - r_2a + br_0b + r_2ab. \quad (18)$$

No commutativity was assumed while checking this identity: r_2 is an explicit source relation. The checker expands all words on both sides and compares coefficients. The run considered 41 columns, retained rank 26, and returned a 475-byte compact certificate.

The exact relation indices and contexts are present in `results/certificates.json`. The accompanying emitter turns this certificate into a generic-ring Lean script, since all eight coefficients are integers. The script first proves (18) by noncommutative normalization and then replaces r_0, r_1, r_2 by zero. The generated file is still an uncompiled target; source generation alone does not establish that its syntax and proof script work with the pinned toolchain.

5.6 Commutators and a small representation change

For $r = xy - yx - 1$, one can prove for every fixed positive numeral n

$$xy^n - y^n x - ny^{n-1} = \sum_{i=0}^{n-1} y^i r y^{n-1-i}. \quad (19)$$

The search found these identities for $n = 2, \dots, 8$. At $n = 8$ it examined 1,793 columns, retained rank 968, and returned eight certificate terms. The identity itself is small even when blind context enumeration is much larger. This is a useful case for learning a checked commutator template after the first few successes, but the stored numeral certificates are not a proof of the universally quantified statement in n .

An important compression opportunity follows from the example. The ordinary identity

$$[x, uv] = [x, u]v + u[x, v]$$

can be proved once and used to propose a linear-size certificate for repeated products. The fallback word-space search remains valuable for discovering a new relation consequence, while a known structural lemma can prevent it from re-enumerating thousands of irrelevant contexts. This is a concrete representation-sensitive route from Forge's existing lemma machinery to the new worker, not a proposal to resaturate distributivity globally.

For a fixed rational q , the relation $xy = qyx$ gives $x^2y^2 = q^4y^2x^2$. Five rational choices, including $q = 1/2$, were tested. A symbolic q would require a coefficient extension with centrality and domain semantics explicitly represented. The current prototype does not infer those conditions or treat a new noncentral letter as a rational parameter.

5.7 Preventing accidental commutativity

Without hypotheses, $xy - yx$ does not vanish in a free algebra. A concrete counterexample is given by the matrix units

$$X = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix} : \quad XY = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}, \quad YX = \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}.$$

Similarly $xy = 0$ does not imply $yx = 0$: take $X = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$ and $Y = \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix}$. The bounded search returns UNKNOWN on both corresponding unsupported goals. The displayed matrices are separate mathematical counterexamples; the worker’s failure alone is not a refutation.

The checker preserves ordered letter tuples throughout. Reversing a word, changing a source relation, introducing an unknown letter, changing a relation index to a Boolean, or replacing a rational coefficient by a nearby rational is rejected when it invalidates the identity. “Normalize the words by sorting them” is never an allowable optimization.

5.8 Coefficients, torsion, and available libraries

A rational certificate naturally applies to a $\mathbb{Q}$ -algebra. An integral certificate applies to an arbitrary associative unital ring, using its central integer scalars. Clearing denominators in a rational certificate may prove $Nf = 0$ in a general ring without proving $f = 0$: cancellation can fail in the presence of torsion. Therefore the emitter accepts only integral certificates for its generic [Ring R] examples. Rational replay needs an actual rational-algebra structure or a separately justified cancellation theorem.

Mathlib’s `linear_combination` is documented for commutative ring expressions; it should not be used as an unexamined noncommutative backend. Small word identities belong with `noncomm_ring`; a larger reflected checker belongs with the semantics of `FreeAlgebra` [15, 14, 16].

Sage exposes free associative algebras via Singular’s letterplace implementation, including homogeneous and weighted-homogeneous methods [17]. This is a candidate search accelerator, not a drop-in solution for the inhomogeneous relation $xy - yx - 1$. A production adapter must select a supported homogenization or a genuinely inhomogeneous method, retain representation witnesses throughout reduction, and dehomogenize with a proof-preserving map. A backend returning only a zero normal form is not enough to feed (15); it must export a trace or the two-sided combination.

The broader literature includes letterplace treatments of nongraded ideals and noncommutative Gröbner computations [18]. None of that functionality was exercised in this run. The delivered bounded linear search is intentionally independent of Singular and works directly with inhomogeneous relation polynomials.

6 A stronger universal-assertion backend for Leant and Djex

6.1 Use the existing candidate stream

Leant’s inspected behavioral interface checks an exact rendered candidate against its type and then attempts the user assertion and its negation using `decide` and bounded `simp`. Its own documentation distinguishes a finite observation from a universal law and treats unsuccessful proof attempts as inconclusive. Djex already exposes an incremental typed stream through the `runDjinnTypedQueryStream` family; its streaming report describes retained candidate ownership, cumulative budgets, and discovery-order delivery [7, 8].

There is no reason to replace either mechanism. Add a stronger proof backend *after* the exact candidate has been fixed. The request consists of that candidate, its original type, the assertion with that candidate substituted, and the same scoped source environment. The backend tries ordinary proof simplification first, then the new invariant or summation worker when a certified extraction is available. Failure still means inconclusive, not that the candidate is wrong.

The point of transfer is specific. A universally quantified assertion about a polynomial accumulator can require an invariant that simplification alone does not invent. A universal assertion about a factorial-weighted sum can require an antidifference. Neither assertion is established by checking a larger finite list of examples. The worker instead supplies the missing mathematical lemma and an induction or finite-sum argument.

6.2 A source-to-state extraction contract

For a candidate recursion over natural numbers, an extractor should return a transition specification together with a Lean theorem connecting the actual candidate to the specified orbit. It is not enough to recognize a familiar function name or print the candidate’s apparent update as a polynomial.

For example, suppose the source candidate computes a pair of accumulators by

$$(a, b) \mapsto (2a, 3b),$$

and a derived output is intended to equal their product. Introduce a third state coordinate for that output, extract its recurrence, and prove that the source program’s state evolves according to the extracted map. An invariant such as $z - ab = 0$ can then prove a universal specification through Theorem 2.1.

A product-program variant compares an implementation and a reference computation. Carry both states and the common input position in one state tuple; initialize them under their actual relation; step them in a certified aligned fashion; and prove that the desired output difference is in the returned invariant span. A returned equality between state coordinates is useful only if the output projection theorems connect those coordinates to the source values.

This first extractor should be syntactically restrictive: structurally recursive natural functions or explicit folds, polynomial state updates over rational or real coordinates, and source equations available as proved lemmas. Integer and natural implementations require checked embedding and operations with compatible semantics. General recursion, higher-order continuations, truncated division, and hidden effects remain opaque unless separately modeled. The delivered Python prototypes consume already extracted problems; they do not implement this Lean compiler pass.

6.3 Uniform inputs: a concrete extension for list folds

A list fold has a new element at each step, not just a choice among finitely many constant transitions. There is a useful extension of the subspace algorithm that handles an arbitrary fresh polynomial input without discretizing it or testing finitely many values.

Let an update be $T(x, e)$, polynomial in state x and input e . Expand each feature pullback in input monomials:

$$m_i(T(x, e)) = \sum_{\beta} (K_{\beta} m_i)(x) e^{\beta}. \quad (20)$$

The finite collection of coefficient maps $K_{\beta} : E \rightarrow \mathbb{Q}[x]$ is linear in the feature polynomial. Replace each branch pullback in (2) by all of its coefficient maps. The same descending-subspace algorithm applies. Its final certificate is

$$p(T(x, e)) = \sum_{\beta} H_{\beta} p(x) e^{\beta} = H(e) p(x). \quad (21)$$

The input is universally quantified afresh at each step, not retained as a single fixed parameter throughout the execution.

Proposition 6.1 (Uniform-input extension). *The coefficient-map construction terminates by dimension descent and proves its invariant family for every finite sequence of input values. If E contains all state monomials of degree at most D and T is affine in the state variables, with polynomial dependence on the input, it is complete for rational state-polynomial equalities of degree at most D that hold under all such input sequences and the specified initial states.*

Proof. For soundness, each coefficient map sends the final space into itself; hence (21) preserves simultaneous vanishing for every input. Induct on the input sequence length.

For completeness, let W be the degree-bounded polynomials vanishing on all reachable states. For every reachable x , the polynomial $p(T(x, e))$ is zero for every input e . Since the coefficient field embeds in an infinite field, each coefficient in e vanishes at that x . Affineness in state variables ensures that each coefficient polynomial still has state degree at most D . Thus every coefficient map preserves W , and the maximality argument of Theorem 2.1 applies. $\square$

This extension is specified and proved here but not implemented in the main corpus. Its implementation is concrete: extract coefficient matrices in the fresh input variables, reuse the existing descent routine, and change each constant transition matrix in the certificate to a finite list of constant matrices tagged by input monomials. Replay expands the resulting polynomial matrix. It is not the unrestricted jointly unknown polynomial-multiplier problem: the input-coefficient maps are fixed by the known transition.

As a small fold example, two accumulators both updated by e^2 preserve $a - b$ uniformly in e . If the reference update is instead $2e^2$, the equality fails except under extra assumptions, which the worker must not invent. This distinguishes universal accumulator reasoning from finite behavioral sampling in exactly the setting where Leant’s assertion interface is useful.

6.4 A proposed narrow proof-service interface

The proposed interface below is new code to implement, not an API claimed to exist in either neighboring project:

```

proveCandidateAssertion(exactCandidate, assertion, sourceContext):
  try ordinary Lean proof simplification
  if a source-linked polynomial orbit specification is available:
    request an inductive-subspace certificate
    prove its polynomial identities in Lean
    combine with the orbit theorem and the assertion projection
  if a source-linked finite sum specification is available:
    request a telescoping certificate and a boundary plan
    prove ratios, guards, exceptions, and the finite range equation
  return a checked assertion proof, or Inconclusive(reason)

```

A theorem from this backend is attached to the exact candidate already being considered. It is not a reason to consume the unobserved tail of Djex’s stream, borrow a receipt from a textually identical candidate, or reset a budget after an inconclusive attempt. Those policies already exist in the inspected code and should remain unchanged [7, 8].

7 Lean integration: concrete modules and library choices

7.1 Implement the replay boundary first

The first deliverable inside Forge should be an ordinary proof-producing adapter for one worker, not a modification of the core `grind` extension state. Forge already discusses both the conservative and invasive integration routes; this report relies on that choice rather than repeating the architecture [5].

A useful new module layout is:

```
Forge/Invariant/FeatureSpace.lean    -- source-linked feature denotation
Forge/Invariant/ClosureReplay.lean  -- init, matrix step, target identities
Forge/Invariant/ExtractOrbit.lean   -- source recursion -> orbit theorem
Forge/Invariant/SearchBridge.lean    -- bounded rational subspace oracle

Forge/Summation/RatioSpec.lean      -- source sequence ratio theorem
Forge/Summation/Antidifference.lean -- rational identity lifting
Forge/Summation/FiniteRange.lean    -- exact finite sum combinators
Forge/Summation/BoundaryPlan.lean   -- exceptional indices and flux extension
Forge/Summation/SearchBridge.lean    -- Gosper/Ore certificate import

Forge/Noncomm/WordPoly.lean         -- ordered-word polynomial semantics
Forge/Noncomm/TwoSidedReplay.lean   -- sum c * left * relation * right
Forge/Noncomm/SearchBridge.lean      -- bounded word-context discovery
```

These paths are proposed additions to Forge; they are not files asserted to exist in the reviewed repository. The accompanying `lean/` directory contains smaller standalone replay targets, not this full module tree.

7.2 What each adapter must prove

For invariants, the source layer supplies an initialization theorem, the transition interpretation, and the target interpretation. The replayer proves each component identity in (4)–(6) with `ring` on the original scalar expressions, constructs simultaneous vanishing, and applies a reachable-state induction theorem. It need not verify the producer’s row reduction or its claim of maximality to accept a positive certificate.

For summation, the source layer supplies the actual ratio recurrence and the finite index set. The replayer checks (8) or (10), proves the denominator guards and exceptional cases, and applies a finite telescoping theorem. The algebraic identity should be stored as an intermediate theorem even when a boundary obligation remains unsolved; it is partial progress, not a completed sum proof.

For noncommutative reasoning, the source layer supplies an ordered variable table and actual relation proofs. A small replayer expands the displayed combination with `noncomm_ring` and rewrites the exact relation subterms. A generic reflected implementation can later evaluate a word-polynomial certificate and apply a soundness theorem over the appropriate algebra. No commutative solver should be allowed to identify words during this replay.

7.3 Concrete dependencies and their roles

Library or mechanism	Specific use	Status in this work
SymPy 1.14.0	Polynomial coefficient matrices; exact nullspace and linear solves; tested <code>hypersimp/gosper_term</code> adapter	Installed and executed
Python <code>fractions.Fraction</code>	Independent sparse commutative and word polynomial replay	Executed, including a fresh <code>python -S</code> process
SymPy <code>DomainMatrix</code>	Rational-domain or fraction-free matrix backend for larger subspace problems	Documented optimization target, not used in timing results [10]
Mathlib <code>ring</code> , <code>finite sums</code> , <code>noncomm_ring</code>	Ordinary proof-term reconstruction of identities and telescoping	Source-level targets; <code>noncomm_ring</code> inspected at the exact pin [14]
Mathlib <code>FreeAlgebra</code>	Semantics for a future word-polynomial reflected checker	Library identified; no compiled checker [16]
Sage <code>ore_algebra</code>	More general telescoper search, rational solutions, and desingularization	Proposed search backend; not installed or executed [13]
Sage/Singular letter-place	Search acceleration for supported free-algebra ideal computations with retained witnesses	Proposed; homogeneous and inhomogeneous modes must not be confused [17, 18]

Recent Lean work already supplies certificate-oriented commutative polynomial tactics for Gröbner reasoning, ideal membership, and related tasks. It should be evaluated as a reusable backend when a future invariant worker needs commutative ideal certificates; it is not evidence that the two-sided noncommutative adapter above already exists [19]. This report does not claim compatibility with that external tactic project or report its performance as its own.

7.4 Generated Lean files and the first acceptance gate

The archive contains a core-only reachable-state induction theorem; ordinary rational identities and a finite telescoping combinator; and two generated noncommutative replay scripts. The generator only emits integral certificates into its generic-ring theorems. These files contain no placeholder axioms or `sorry`, but they were not compiled in this environment. Their current status remains `NOT_COMPILED`, not “probably verified.”

Before admitting any worker into Forge, run the real pinned Lake project, compile these files, inspect the axiom inventories, and exercise the replayer on negative controls that modify the source problem rather than just the certificate. Then add a source expression reifier and repeat the controls on actual Lean goals. A Python certificate checked for a manually prepared polynomial problem does not test that reifier.

The first public tactic syntax can be explicit rather than magical. For example, a diagnostic command could show a source-linked transition model, selected features, and a proposed invariant before the full automatic planner chooses them. The eventual `forge` front end can call the same interface silently. This keeps the first capability test focused on mathematical coverage instead of conflating it with complete automation of extraction and scheduling.

8 Executed experiments and what they establish

8.1 Environment and method

The recorded run used Python 3.13.5, SymPy 1.14.0, and seed 20260915 on the Linux environment identified in `results/summary.json`. No Lean or Lake executable was available. No stock `grind`, `simp`, `ring`, or `noncomm_ring` benchmark was run. Existing repository test results were not added to these counts.

The experiment script constructs each problem independently of the certificate returned by search, invokes the checker with both arguments, and saves them in separate files. After generation, a fresh Python process runs the checker with `-S`, which excludes automatic site-package initialization. The replay uses only the standard library and imports neither SymPy nor the producer. This confirms a useful dependency separation; it does not turn Python into a proof kernel.

Search timing covers the individual search call on an already constructed symbolic problem, including coefficient-system construction within that call. It excludes Python/SymPy import startup, corpus generation, mutation tests, serialization, the conservation ablation, and any Lean work. Replay timing covers one in-memory certificate check. Each number is one observation, not a statistical latency estimate.

Worker	Certificates	Search (ms)	Replay (ms)
Inductive subspaces	24	216.1	7.59
One-variable antidifferences	15	247.0	2.19
Creative telescoper	1	107.9	1.59
Two-sided word relations	25	110.5	4.24
Total	65	681.4	15.60

Table 2: Recorded single-run totals for the stored positive corpus. Search and replay are Python operations on prepared problems, not Lean tactic timings. The two summation rows certify algebraic identities only.

8.2 The positive corpus

The 24 invariant cases consist of six named examples and 18 planted affine families. The named cases are the scaling graph, the mutually preserved pair, the square counter, a rotation preserving a circle, two equal accumulators, and a two-branch scaling graph. The planted families initialize $z = x + y$ and construct two affine transitions preserving that relation with a scalar factor different from one. These are mechanism tests chosen to distinguish the two certificate languages, not a representative sample of arbitrary Lean proofs.

The 15 one-variable summation cases consist of a factorial-weighted term, two reciprocal terms, six geometric terms, and six weighted geometric terms. The sole creative-telescoping instance is the squared-binomial example. The 25 noncommutative cases consist of seven Weyl-commutator numeral instances, five rational q -commutation instances, the commuting-idempotent join, and 12 planted two-sided combinations.

All 65 returned certificates passed independent replay. Their acceptance means exact closure or algebraic identities for the explicitly supplied problem objects. For the summation kinds it does *not* include a source sequence bridge or universally checked boundary plan.

8.3 A same-feature invariant ablation

The reference ablation uses the same feature degree, initial map, branch maps, and target, but restricts its candidate space to

$$p \circ I = 0, \quad p \circ T_b - p = 0 \quad \text{for every branch.}$$

It solves these exact coefficient constraints directly. This is an implementation of the *conservation-only certificate language*, not a rerun of an original Forge executable.

On the 24 selected cases the new subspace algorithm proves target membership in 24 cases and the conservation-only ablation in three. The latter succeeds on the square counter, circle rotation, and equal accumulators. The other examples were intentionally selected or planted to require scaling or collective preservation. Thus the observed gap demonstrates added expressive coverage, not a 24-to-3 estimate for general proof workloads or a performance victory over Lean’s native tactics.

For the scaling graph the gap can be explained mathematically, independently of the experiment. In degree at most two, the initialization kernel is the one-dimensional span of $xy - z$. Pullback acts on that span as multiplication by six. Its only fixed vector is zero, whereas the whole line is invariant. No better implementation of the same conservation-only ansatz can recover the missing target in that feature space.

8.4 Negative and adversarial controls

Seven unsupported cases are recorded as UNKNOWN: three invariant cases, two bounded antidifference failures, and two noncommutative consequence failures. These outcomes are never interpreted as global impossibility certificates. The nonlinear degree-escape case is especially important because the desired invariant is true; it checks that the report distinguishes fragment incompleteness from a false goal.

The 188 certificate mutations are assembled from uniform and targeted controls. Every stored certificate is tested with its essential payload removed and with an unexpected field inserted. Every invariant certificate is also tested with a different target and with a missing transition matrix. Additional cases change a branch, zero a denominator, introduce a floating-point or noncanonical coefficient, duplicate a monomial, allow a recurrence coefficient to depend on k , change a source relation, misuse a Boolean as an index, introduce an unknown letter, and make a tiny exact rational change. Every tested invalid variant is rejected.

These tests do not establish that the decoder is secure against every hostile byte stream. It checks field sets, rational string canonicity, dimensions, monomial ordering, bounded indices, degree and size caps, and a product-work budget. It still relies on Python’s parser and allocation behavior, and it is not fuzzed across all possible nested JSON inputs. The code is a research checker with explicit limits, not a hardened network service.

8.5 Independent arithmetic and boundary checks

One hundred randomized substitution tests compare the checker’s sparse substitution with SymPy’s simultaneous substitution and expansion. These exercise a different algorithmic path from accepting a stored solver result. They reduce the risk of a shared substitution convention error; they are not an exhaustive arithmetic proof.

For the squared-binomial flux, 527 concrete (n, k) pairs cover $0 \leq n \leq 30$ and $0 \leq k \leq n + 1$. The experiment also checks the finite sum and the three relevant flux values for every one of those 31 n values. The $n = 0$ case and both singular strata occur in this range. Additional factorial-sum checks use $1 \leq a \leq 8$ and $a \leq b \leq 11$, including empty ranges. None of these samples is used as the acceptance condition for a polynomial identity.

The generated run records 1,100 assertions in total; this count includes individual samples, positive checks, mutation rejections, dimension checks, and the fresh-process replay. It is *not* a count of 1,100 independent universal theorems. The separate standard `unittest` suite has 13 named test methods, with four SymPy-adapter inputs inside one method and stored certificate subtests inside another. Their raw log is included.

8.6 What has not been tested

No result here measures automatic Lean term extraction, dependent typing, proof-term size inside the kernel, equality transport, source cast handling, the performance of a real Forge controller, or the behavior of a compiled `grind` plugin. The uncompiled Lean files do not fill those gaps. There is also no large benchmark demonstrating that the chosen context or feature bounds are sensible on typical formalization tasks.

A useful next comparison would separate three conditions: a raw goal, a goal with the worker’s discovered mathematical lemma supplied, and a goal with a complete certificate supplied. Compare each against the pinned baseline under identical theorem inventories and trust settings. That comparison can reveal whether the value lies in discovery, local proof reconstruction, or cross-worker cooperation. The present experiment establishes the first component’s possibility on explicit problems, not the whole system’s superiority.

9 Implementation sequence and acceptance criteria

9.1 First increment: noncommutative relation replay

This is the smallest isolated production addition. Its semantic objects are words, rational or integer coefficients, relation references, and a target. Start with integral certificates in associative rings and small source expressions. Require compilation of the emitted examples, a checked source reifier preserving word order, and negative tests where ab and ba differ. Then connect the existing bounded producer and expose the resulting theorem to Forge. Add rational coefficients only with an explicit algebra structure.

This ordering is not a prediction that the noncommutative worker has the largest eventual coverage. It has the shortest path from an already executed certificate to a useful Lean theorem, with relatively little domain-specific boundary work.

9.2 Second increment: source-linked inductive subspaces

Implement the generic simultaneous-invariant induction theorem and replay small closure matrices using ordinary polynomial normalization. Support a single source-linked natural recursion or an explicit transition relation before general list-fold extraction. Acceptance requires the scaling example, a mutually preserved family, multiple branches, a genuine target projection, and failures when the initialization or a branch is changed.

Only after these pass should feature selection become automatic. Add the uniform-input coefficient-map extension of Section 6.3 to cover polynomial list folds without finite-alphabet sampling. Add multi-location control-flow spaces separately; neither feature requires changing the positive-certificate theorem into a global completeness claim.

9.3 Third increment: a small summation library

Prove the finite-range telescoping combinator, the guarded antidifference lifting theorem, and ratio lemmas for a deliberately short list of source constructors. Integrate the tested SymPy Gosper adapter through typed data, not by importing executable code or an unproved symbolic equality. Require tests at zero, at empty ranges, at denominator zeros, and at parameter values where a recurrence coefficient vanishes.

The first complete creative-telescoping acceptance target should be the squared-binomial theorem with the nonsingular flux, because it exercises the hard boundary mechanism rather than only an interior rational identity. General Ore-algebra import comes later, after the adapter can explain exactly which returned object becomes which Lean theorem.

9.4 Stopping rules that preserve value

Each worker should be independently useful before the next is started. The noncommutative worker can prove a relation consequence without induction. The invariant worker can export an equality useful to `grind` or an arithmetic worker. The summation worker can produce a checked local antidifference even when a more ambitious closed-form goal needs additional steps.

When a bound is reached, retain any already checked consequences and report UNKNOWN for the remaining request. Do not infer that a theorem is false because a feature space, denominator list, or context dictionary failed. Do not increase a bound indefinitely when the failure has a structural explanation, such as nonlinear degree escape or a singular endpoint. Those explanations are routing information for Forge’s existing planner.

10 Conclusion

The strongest next version of Forge need not acquire another general-purpose search slogan. It can acquire three concrete mathematical capabilities: find a space of equations that evolves into itself, find a local flux whose finite sum gives a recurrence, and find a two-sided combination of noncommutative relations. Their certificates are small, their checking conditions are explicit, and their failure modes point to meaningful alternative workers.

The delivered prototypes establish that these searches can be executed and that their proposed identities survive independent exact replay. The invariant ablation demonstrates a real increase in the chosen certificate language’s coverage. The binomial example demonstrates why source semantics cannot be collapsed into polynomial checking. The noncommutative examples demonstrate how to discover useful consequences without falsely assuming commutativity.

The remaining work is substantial but concrete: source-linked Lean extraction, actual kernel replay, and fair end-to-end evaluation. Those are engineering acceptance gates, not reasons to replace the algorithms with a vague proposal.

A Certificate formats and reproducibility

A.1 Canonical arithmetic data

A commutative polynomial is a sorted list of pairs $\{\mathbf{m}, c\}$. The field $\mathbf{m}$ is an exponent vector of fixed arity; c is a canonical rational string such as `"-3/2"`. Zero coefficients and duplicate monomials are forbidden. A noncommutative polynomial uses the same external shape, but $\mathbf{m}$ is an ordered word of letter indices. The `kind` tag fixes the interpretation; a word is never decoded as a commutative exponent vector.

An invariant certificate contains the polynomial basis, one rational matrix per source transition, and coefficients expressing the source target in that basis. The initial map and all transitions live in the source problem file, not in the producer's certificate. The scaled-graph certificate has basis $xy - z$, matrix `[6]`, and target coefficient `-1`.

A Gosper certificate contains only U, V ; the source problem supplies A, B . The creative certificate adds c_0, c_1 and uses the source pair of ratio numerators and denominators. A two-sided certificate contains terms with `relation`, `left`, `right`, and `coefficient` fields. The source relation array and target are not replaceable by those terms.

The checker permits no JSON floating-point literals, duplicate object keys, unknown certificate fields, noncanonical rational strings, or invalid monomial orderings. It limits source dimensions to eight, polynomial degree to 32, expanded terms to 20,000, and elementary multiplication-pair work to two million per certificate. These are conservative operational rejection limits; changing them does not change the mathematical soundness argument.

A.2 Archive contents

Path	Purpose
<code>article/forge_extensions.tex</code> and <code>article/forge_extensions.pdf</code>	This report and its editable source
<code>prototype/search.py</code>	Untrusted exact search, including the optional SymPy Gosper adapter
<code>prototype/checker.py</code>	Standard-library-only independent replay
<code>prototype/experiments.py</code>	Deterministic corpus, ablation, mutations, differential checks, and result capture
<code>prototype/test_regressions.py</code>	Thirteen named regression methods
<code>prototype/emit_lean.py</code>	Integral noncommutative replay-script emitter
<code>results/</code>	Source problems, certificates, per-case measurements, summary, expected unknowns, assertions, mutations, and raw logs
<code>lean/</code>	Pinned project targets and uncompiled ordinary Lean proofs
<code>docs/review-scope.md</code> and <code>docs/status.json</code>	Source pins, nonduplication scope, and machine-readable evidence status

A.3 Commands

From the archive root, replay the stored certificates without third-party packages:

```
python -S prototype/checker.py
```

Install the search dependency and run the named tests:

```
python -m pip install -r prototype/requirements.txt
python -m unittest discover -s prototype -p 'test_*.py' -v
```

Regenerate the experiment into a fresh directory:

```
python prototype/experiments.py --output reproduced-results
python -S prototype/checker.py reproduced-results/problems.json \
    reproduced-results/certificates.json
```

The generator refuses a nonempty output directory, so the recorded results are not silently overwritten. A new run can have different timings while producing the same exact certificates at the recorded dependency version. Build this report with two pdfL^AT_EX passes or with `latexmk`:

```
cd article
latexmk -pdf -interaction=nonstopmode -halt-on-error forge_extensions.tex
```

The separate `lean/README.md` describes the unexecuted Lean build and axiom-inspection steps. No `lake-manifest.json` is fabricated for a build that did not occur.

B A compact set of regression targets for a Lean port

A port should preserve the following distinctions before it is called an implementation of these workers.

Invariant positive target. Starting at (u, v, uv) and applying $(x, y, z) \mapsto (2x, 3y, 6z)$ any number of times preserves $z = xy$. The initial parameters are universally quantified. Changing the third update to $6z + 1$ must invalidate the same certificate.

Collective closure target. Starting at $(0, 0)$ and applying $(x, y) \mapsto (y, 2x)$ preserves both coordinates' vanishing. A replay that checks only one diagonal entry of the matrix must not pass.

Nonlinear incompleteness target. Starting at zero under $x \mapsto x^2$ preserves $x = 0$, but the finite constant-closure subspace search returns UNKNOWN. The tactic may use another proof route; it must not call the theorem false.

Antidifference guard target. The factorial-weight multiplier $1/k$ is valid on the intended positive indices. A proof that substitutes $k = 0$ without cancellation or a separate extension theorem is invalid.

Creative boundary target. The squared-binomial certificate must prove the two exceptional strata $k = n$ and $k = n + 1$, not just the generic rational identity. The $n = 0$ case has no generic interior and must still be covered.

Word-order target. Given $ab - ba = 1$, prove the fixed numeral commutator consequence. With no relation, do not prove $ab = ba$. Given only $ab = 0$, do not prove $ba = 0$.

Coefficient-domain target. Replay an integral two-sided certificate in an arbitrary associative ring. Require the appropriate rational scalar structure before replaying a genuinely rational one; do not cancel a cleared integer denominator in a torsion ring.

Exact-candidate target. A universal behavioral proof must concern the same source term that was type-checked by Leant. Failure of the new backend is inconclusive and does not consume a success slot or establish noninhabitation.

References

- [1] Vladimir Reshetnikov. *Forge*, README and repository tree. Revision 674521027d968d59f7b83220ed52304a85cb55e2, inspected September 15, 2026. <https://github.com/VladimirReshetnikov/Forge/tree/674521027d968d59f7b83220ed52304a85cb55e2>.
- [2] Forge, *Structural search section*. `article/sections/05-structural.tex`, same revision as [1].
- [3] Forge, *Nonlinear arithmetic section*. `article/sections/06-nonlinear.tex`, same revision as [1].
- [4] Forge, *Boolean search, equality exploration, and external engines*. `article/sections/07-boolean-and-external.tex`, same revision as [1].
- [5] Forge, *Lean integration: a conservative bridge before an invasive one*. `article/sections/08-integration.tex`, same revision as [1].
- [6] Forge, *Provenance: what came from where*. `article/sections/B-provenance.tex`, and `docs/IDEAS-FROM-LEANT-DJEX.md`, same revision as [1].
- [7] Vladimir Reshetnikov. Leant, *Behavioral assertions during synthesis*. `docs/behavioral-synthesis.md`, revision 6bf05ad78c467989e68290f2d08bbbed40802d485. <https://github.com/VladimirReshetnikov/Leant/blob/6bf05ad78c467989e68290f2d08bbbed40802d485/docs/behavioral-synthesis.md>.
- [8] Vladimir Reshetnikov. Djex, *Incremental Djinn behavioral synthesis*. Report dated September 6, 2026, revision e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5. <https://github.com/VladimirReshetnikov/Djex/blob/e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5/docs/reports/2026-09-06-djinn-behavioral-streaming.md>.
- [9] Ehud Hrushovski, Joël Ouaknine, Amaury Pouly, and James Worrell. *Polynomial invariants for affine programs*. LICS 2018, pp. 530–539. DOI: 10.1145/3209108.3209142. <https://arxiv.org/abs/1802.01810>.
- [10] SymPy Development Team. *Introducing the DomainMatrix of the poly module*. SymPy 1.14.0 documentation. <https://docs.sympy.org/latest/modules/polys/domainmatrix.html>.
- [11] SymPy Development Team. *Concrete mathematics*, including `hypersimp`, `gospers_normal`, `gospers_term`, and `gospers_sum`. SymPy 1.14.0 documentation. <https://docs.sympy.org/latest/modules/concrete.html>.
- [12] Mathlib contributors. *Mathlib.Data.Finset.Range*. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Data/Finset/Range.html.
- [13] Manuel Kauers and collaborators. *ore_algebra*: Sage implementation of Ore algebras, Ore polynomials, and differentially finite functions. Official Sage package documentation and project. https://doc.sagemath.org/html/en/reference/spkg/ore_algebra.html; https://github.com/mkauers/ore_algebra.

- [14] Jireh Loreaux, Kim Morrison, Oliver Nash, and Mathlib contributors. *Mathlib.Tactic.NoncommRing*. Source inspected at 1cf325a0cf67aca2b04d76b5380ff6a9e410aefa. <https://github.com/leanprover-community/mathlib4/blob/1cf325a0cf67aca2b04d76b5380ff6a9e410aefa/Mathlib/Tactic/NoncommRing.lean>.
- [15] Mathlib contributors. *Mathlib.Tactic.LinearCombination*. Official generated documentation. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/LinearCombination.html.
- [16] Mathlib contributors. *Mathlib.Algebra.FreeAlgebra*. Official generated documentation. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Algebra/FreeAlgebra.html.
- [17] SageMath contributors, including Simon King. *Free associative unital algebras, implemented via Singular's letterplace rings*, and associated element documentation. Official Sage reference. https://doc.sagemath.org/html/en/reference/algebras/sage/algebras/letterplace/free_algebra_letterplace.html.
- [18] Roberto La Scala. *Extended letterplace correspondence for nongraded noncommutative ideals and related algorithms*. 2012. <https://arxiv.org/abs/1206.6027>.
- [19] Hao Shen, Junyu Guo, Junqi Liu, and Lihong Zhi. *Automated Tactics for Polynomial Reasoning in Lean 4*. April 15, 2026, arXiv:2604.13514v1. <https://arxiv.org/html/2604.13514v1>.